National University of Singapore

CS1101S — Programming Methodology

AY2025/2026 Semester 1

Final Assessment

Time allowed: 2 hours

QUESTIONS

INSTRUCTIONS

1. This **QUESTION PAPER** contains **24** Questions in **8** Sections, and comprises **21** printed pages, including this page.
2. The **ANSWER SHEET** comprises **8** printed pages.
3. Use a pen or pencil to **write** your **Student Number** in the designated space on the front page of the **ANSWER SHEET**, and **shade** the corresponding circle **completely** in the grid for each digit or letter. DO NOT WRITE YOUR NAME!
4. You must **submit only** the **ANSWER SHEET** and no other documents. Do not tear off any pages from the ANSWER SHEET.
5. All questions must be answered in the space provided in the **ANSWER SHEET**; no extra sheets will be accepted as answers.
6. Write legibly with a **pen** or **pencil** (do not use red color). Untidiness will be penalized.
7. For **multiple choice questions (MCQ)**, **shade** in the **circle** of the correct answer **completely**.
8. The full score of this assessment is **100** marks.
9. This is a **Closed-Book** assessment, but you are allowed to bring with you one double-sided **A4 / letter-sized sheet** of handwritten or printed **notes**.
10. Where programs are required, write them in the **Source §4** language. A **reference** of some of the **pre-declared functions** is given in the **Appendix** of the Question Paper.
11. In any question, unless it is specifically allowed, your answer **must not use functions** given in, or written by you for, other questions.

Section A: Higher Order Functions [12 marks]

(1) [4 marks]

Implement the function `map_if` that takes a predicate `pred`, a unary function `f` and a list `xs`, and returns a new list. This new list contains the same elements as `xs`, except that every element x for which `pred`(x) is true is replaced by `f` (x). For example,

```
const xs = list(1, 2, 3, 4, 5);
const new_xs = map_if(x => x % 2 === 0, x => 2 * x, xs);
display_list(new_xs); // displays list(1, 4, 3, 8, 5)
```

Write your answer only in the dashed-line box(es).

(2) [4 marks] (MCQ)

What is the result of evaluating the following Source program?

```
const compose = (f, g) => x => f(g(x));
const inc = x => x + 1;
const double = x => x * 2;
const square = x => x * x;
const h = compose(compose(double, inc), square);
h(3);
```

A.	`18`
B.	`19`
C.	`20`
D.	`37`
E.	`40`
F.	`64`
G.	None of the options is correct

(3) [4 marks] (MCQ)

Consider the following Source program.

```
function make(a) {
  return b => {
    const a = b + 1;
    return c => a + b + c;
  };
}

const p  = make(10);
const r1 = p(2)(3);
const r2 = make(7)(3)(1);
```

What are the values of `r1` and `r2`?

A.	`r1: 5  r2: 11`
B.	`r1: 6  r2: 8`
C.	`r1: 6  r2: 10`
D.	`r1: 8  r2: 8`
E.	`r1: 8  r2: 10`
F.	`r1: 10  r2: 10`
G.	None of the options is correct

Section B: Tagged Functions [13 marks]

For the following questions in this section, we will represent functions together with their type information using tagged lists. A **tagged function** is a list of three items:

```
list(f, input_type, output_type);
```

where `f` is a unary function, `input_type and output_type` are strings describing the types of the function `f`.

For example:

```
const add = list(x => x + 1, "number", "number");
const to_string = list(x => stringify(x), "number", "string");
```

You can access these components of the tagged function with the following functions:

```
const tf_f   = tf => head(tf);                // returns function
const tf_in  = tf => head(tail(tf));          // returns input type
const tf_out = tf => head(tail(tail(tf)));    // returns output type
```

We can compose tagged functions in a chain of computation, as a list of tagged functions. For example,

```
const chain = list(list(x => x + 1, "number", "number"),
                   list(x => x * 2, "number", "number"),
                   list(x => x + 7, "number", "number"));
```

When this chain of functions `chain` is applied to the value `5`, the intended computation is `((5 + 1) * 2) + 7 = 19`.

(4) [5 marks]

Implement the function `apply_tagged` takes in a chain of computation `chain` and a value `x` and returns the result of recursively applies all functions in `chain` to input `x`, from left to right, returning the final value. Do not use loops in your answer. For example,

```
apply_tagged(chain, 5); // returns 19
apply_tagged(null, 5);  // returns 5
apply_tagged(chain, 6); // returns 21
```

Write your answer only in the dashed-line box(es).

(5) [8 marks]

Implement the function `type_check` which takes in a chain of tagged functions `chain` and a string `start_type`, and checks whether the chain is **well-typed**. A chain is well-typed if the output type of each function matches the input type of the next function, starting from `start_type`. If the entire chain is well-typed, `type_check` should return `true`; otherwise, it should return `false`. For example,

```
const ok_chain = list(
  list(x => x + 1, "number", "number"),
  list(x => stringify(x), "number", "string")
);

const bad_chain = list(
  list(x => x + 1, "number", "number"),
  list(s => s + "!", "string", "string")
);

type_check(ok_chain, "number");  // returns true
type_check(bad_chain, "number"); // returns false
type_check(null, "number");      // returns true
```

Write your answer only in the dashed-line box(es).

Section C: The Hungry C*der's Canteen [25 marks]

A canteen has several **stalls**, each identified by a *stall number* (starting from 0).
Each stall maintains a **queue of pending orders**, represented as a list of dish names.
The canteen itself is a **list of such queues**, one for each stall.

(6) [3 marks]

Complete the implementation of the following `create_canteen(num_stalls)` function, which takes in the number of stalls as input and creates a list of `num_stalls` empty lists representing empty queues at the start of the canteen's business day.

```
// example 1
const canteen1 = create_canteen(3);
display_list(canteen1);      // list(null, null, null)
```

(7) [3 marks] (MCQ)

Complete the implementation of the following `set_stall(canteen, stall_num, new_queue)` function, which takes in a canteen, stall number and a new queue as inputs and sets the new queue as the order queue for the given stall number.

```
function set_stall(canteen, stall_num, new_queue) {
    if (stall_num === 0) {
        <ST1>
    } else {
        <ST2>
    }
}
```

Complete the implementation by filling in `ST1` and `ST2`.

A.
```
<ST1>: pair(new_queue, tail(canteen));
<ST2>: set_stall(tail(canteen), stall_num - 1, new_queue);
```

B.
```
<ST1>: new_queue;
<ST2>: pair(head(canteen),
            set_stall(tail(canteen), stall_num - 1, new_queue));
```

C.
```
<ST1>: set_head(canteen, new_queue);
<ST2>: pair(head(canteen),
            set_stall(tail(canteen), stall_num - 1, new_queue));
```

D.
```
<ST1>: set_tail(head(canteen), new_queue);
<ST2>: set_stall(tail(canteen), stall_num - 1, new_queue);
```

E.
```
<ST1>: set_head(canteen, new_queue);
<ST2>: set_stall(tail(canteen), stall_num - 1, new_queue);
```

F.
```
<ST1>: set_tail(canteen, new_queue);
<ST2>: set_stall(head(canteen), stall_num - 1, new_queue);
```

G. None of the other options are valid

(8) [4 marks]

Complete the implementations of `add_to_queue(canteen, stall_num, order)` and `serve_customer(canteen, stall_num).` The `add_to_queue` function adds a new order to the back of the queue for the given stall number. The `serve_customer` function simulates fulfilling and order, returns the customer order and removes the order from the front of the queue for the stall number given.

```
// example 1 continued …
add_to_queue(canteen1, 1, "chicken rice");
add_to_queue(canteen1, 1, "rendang");
display_list(canteen1);
                    // list(null, list("chicken rice", "rendang"), null)
serve_customer(canteen1, 1);
display_list(canteen1); // list(null, list("rendang"), null)
```

Note: Assume you have a selector `get_stall_queue(canteen, stall)` Which takes in a canteen and a stall index as input and returns the queue for that stall. (first stall has index 0)

(9) [3 marks] (MCQ)

You are given the following implementation of `count_occurrences`, which counts the number of occurrences of `x` in a given list `xs`.

```
function count_occurrences(x, xs) {
    return is_null(xs)
        ? 0
        : <P>
        ? <C>
        : <A>;
}
```

A.
```
<P>: head(xs) === x
<C>: 1+count_occurrences(x,tail(xs))
<A>: count_occurrences(x,tail(xs))
```

B.
```
<P>: head(xs)
<C>: count_occurrences(x,tail(xs))
<A>: 1+count_occurrences(x,tail(xs))
```

C.
```
<P>: tail(xs) === x
<C>: 1+count_occurrences(x,xs)
<A>: count_occurrences(x,tail(xs))
```

D.
```
<P>: head(xs) === x
<C>: count_occurrences(x,xs)
<A>: 1+count_occurrences(x,tail(xs))
```

E.
```
<P>: is_null(tail(xs))
<C>: 1
<A>: count_occurrences(x,tail(xs))
```

F.
```
<P>: head(xs) === x
<C>: 1+count_occurrences(x,xs)
<A>: count_occurrences(x,xs)
```

G.
```
<P>: head(xs) !== x
<C>: count_occurrences(x,tail(xs))
<A>: 1+count_occurrences(x,tail(xs))
```

H.
```
<P>: head(xs) === x
<C>: 1+count_occurrences(x,xs)
<A>: 1+count_occurrences(x,tail(xs))
```

(10) [3 marks] (MCQ)

You are given the function `remove_duplicates`, which removes the duplicates of a given list.

```
function remove_duplicates(xs) {
    return is_null(xs)
        ? null
        : pair(head(xs),
               remove_duplicates(
                   filter(x => x !== head(xs), tail(xs))
               ));
}
```

Given that `n = length(xs)`, what is the runtime of this function?

A. Θ(log n)
B. Θ(n)
C. $\Theta(n^2)$
D. Θ(n log n)
E. $\Theta(n^3)$
F. $\Theta(2^n)$
G. Θ(n!)
H. None of the other options is the correct answer

(11) [6 marks]

Write a function `find_most_popular_dish(canteen)`, for a given canteen, find the dish that appears the most frequently in all queues.

Note 1: a dish might be offered in multiple stalls
Note 2: You can assume you have access to correct implementations of any of the previous functions in this section
Note 3: If there are ties – you can respond with any one of the most queued dish

```
// [new] example 2
const canteen2 = create_canteen(2);
display_list(canteen2); // list(null, null)
join_queue(canteen2, 1, "chicken rice");
join_queue(canteen2, 1, "chicken rice");
join_queue(canteen2, 1, "chicken rice");
join_queue(canteen2, 0, "rendang");
join_queue(canteen2, 0, "chicken rice");
join_queue(canteen2, 1, "rendang");
display_list(canteen2);
  //list(list("rendang", "chicken rice"),
      //list("chicken rice", "chicken rice", "chicken rice", "rendang"))

find_most_popular_dish(canteen2); // “chicken rice”
```

(12) [3 marks] (MCQ)

What is the run time of the `find_most_popular_dish` program?
Note: Assuming the program is implemented following the algorithm outlined in the comments for you to fill in the blanks.

N = length(canteen)
M = max queue length

Select the tightest possible upper bound from the given options.

A. $O(N + M)$
B. $O(NM)$
C. $O(N^2 + M)$
D. $O(N^2 + M^2)$
E. $O(NM^2)$
F. $O(N^2M)$
G. $O(N^2M^2)$
H. None of the other options is the correct answer

Section D: Sieve of Eratosthenes [14 marks]

SICP JS describes the ancient Sieve of Eratosthenes in the context of stream processing.

Unfortunately, the stream-based Sieve of Eratosthenes in SICP JS does not match this ancient algorithm. Let us start from scratch and consider the following description attributed to the Greek mathematician Eratosthenes to find all prime numbers up to a certain limit.

(1) Create a collection of consecutive integers from 2 up to the limit. Cross out 0 and 1, because they are not considered prime.
(2) Find the next not-crossed-out number, p, which must be a prime number. (Initially p is 2.)
(3) Cross out multiples of p in the collection, starting from p^2. (Any multiple of p less than p^2 would have been crossed already. So, when p is 2, we cross out 4, 6, 8, etc.)
(4) Repeat the steps from (2) until p is greater than the square root of the limit.
(5) All non-crossed-out numbers in the collection will be prime.

Melissa E. O'Neill points out in her paper "The Genuine Sieve of Eratosthenes"—published in 2009 in Journal of Functional Programming, 19, 95-106—that streams are not very suitable for implementing this algorithm because it requires random access to the collection.

Instead, let us do it using arrays as follows:

```
function sieve_of_eratosthenes(limit) {
    // make array of true values with indices 0 to limit
    const is_candidate = make_array_of_true(limit);

    // 0 and 1 are not prime
    is_candidate[0] = false;
    is_candidate[1] = false;

    // run sieve algorithm
    sieve(is_candidate, limit);

    // make array of indices of remaining candidates
    return indices_of_true(is_candidate);
}

sieve_of_eratosthenes(11); // returns an array of prime numbers up to 11:
                           // [2, 3, 5, 7, 11]
```

In the questions of this section, you are requested to provide the functions `make_array_of_true`, `sieve`, and `indices_of_true`.

(13) [4 marks] The first helper function `make_array_of_true` in `sieve_of_eratosthenes` creates an array of true values whose indices range from 0 to the given limit. Example:

```
make_array_of_true(4); // returns [true, true, true, true, true]
```

Complete the following implementation. No helper functions are allowed.

(14) **[6 marks]** Write a function `sieve` to implement the Sieve of Eratosthenes above. No helper functions are allowed.

(15) **[4 marks]** Write a function `indices_of_true` that takes an array of Boolean values as argument and returns an array that contains all indices at which the given array has the value true. Example:

```
indices_of_true([false, true, true, false, false, true]);
// returns: [1, 2, 5]
```

No helper functions are allowed.

Section E: Reversible Computing [16 marks]

As conventional computing approaches fundamental physical limits, *reversible* computing offers a promising path forward. According to Landauer's principle, *erasing* information requires energy and generates heat—a major constraint for building faster, more efficient computers. Reversible computing avoids information erasure, allowing computations to proceed without this energy cost.

In reversible computing, we distinguish between operations that erase information (irreversible, expensive) and those that don't (reversible, cheap). Modern reversible computing architectures use special reversible counter circuits that can increment without erasing information by using additional ancilla bits. For this question, assume loop counters are implemented this way and focus on array operations.

To account for these characteristics, we define a resource function r(n) for a given algorithm that **counts the number of times a number in an array is overwritten by a new number**. No other operations in any given algorithm is counted in r because their runtime is considered negligible in reversible computing architectures.

(16) [2 marks] (MCQ)

Consider the insertion sort algorithm:

```
function swap(A, x, y) {
    const temp = A[x];
    A[x] = A[y];
    A[y] = temp;
}
function insertion_sort(A) {
    const len = array_length(A);
    for (let i = 1; i < len; i = i + 1) {
        let j = i - 1;
        while (j >= 0 && A[j] > A[j + 1]) {
            swap(A, j, j + 1);
            j = j - 1;
        }
    }
}
```

What is the order of growth of the resource function r(n) for insertion sort where n is the length of the **given array that is sorted in reverse order**.

A) $\Theta(0)$
B) $\Theta(n)$
C) $\Theta(n^2)$
D) $\Theta(n \log n)$
E) $\Theta(\log n)$

(17) **[4 marks]** Let us design a reversible sorting algorithm. The first step is to determine where each element should go in the sorted array. For this, we want to count how many positions in an array A have elements that are strictly smaller than the element at a given index.

Example: If `A is [5, 2, 8, 9, 2]` and `i is 2`, then `count_smaller(A, i)` should return `3` because the three positions `0`, `1`, and `4` have elements that are smaller than `A[i]`.

Complete the implementation of `count_smaller`.

(18) **[5 marks]** Let us use the `count_smaller` helper function from the previous question to complete the first pass of our reversible sort. This pass places each element of a given unsorted array A into a new array R at the position determined by how many positions of A have elements that are smaller than it. **Make sure your algorithm does not erase any number value in A.**

Example: The call `reversible_sort_first_pass([5, 2, 8, 9, 2])` should return the array `[2, undefined, 5, 8, 9]` because in the result array R
2 is in position 0 (0 positions in A have elements smaller than 2),
5 is in position 2 (2 positions in A have elements smaller than 2),
8 is in position 3 (3 positions in A have elements smaller than 2), and
9 is in position 4 (4 positions in A have elements smaller than 2).

No helper functions allowed.
Complete the implementation of `reversible_sort_first_pass`.

(19) **[3 marks]** After the first pass, some positions in `R` remain undefined (ancillae). The overall sorting algorithm works as follows:

```
function reversible_sort(A) {
    const R = reversible_sort_first_pass(A);
    reversible_sort_second_pass(R);
    return R;
}
```

Complete the function `reversible_sort_second_pass` to fill the array appropriately. **Make sure your algorithm does not erase any number value in `R`.**

(20) **[2 marks] (MCQ)**

What is the order of growth of r(n) for `reversible_sort(A)` where `n` is the size of `A`.

A) $\Theta(n^2)$
B) $\Theta(n \log n)$
C) $\Theta(\log n)$
D) $\Theta(n)$
E) $\Theta(0)$

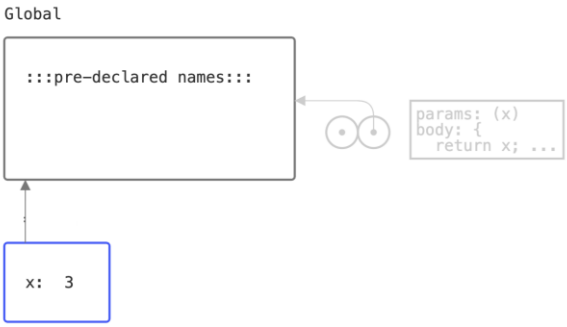
Section F: Environment Model and CSE Machine [6 marks]

(21) [3 marks] (MCQ) Consider the following Source program

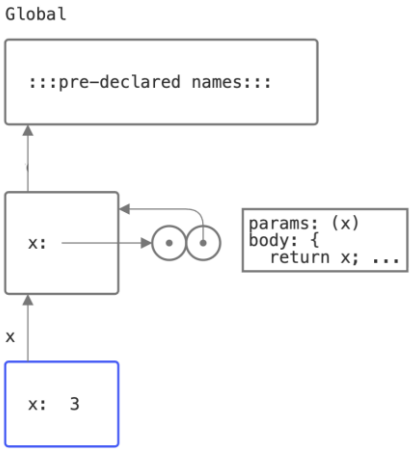
```
(x => x)(3);
```

Which of the following environment diagrams depicts the environment at the end of executing this program?

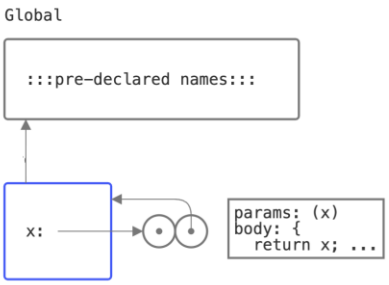
A)



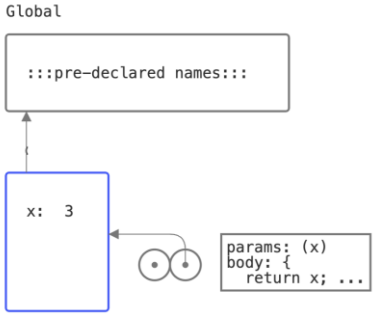
B)



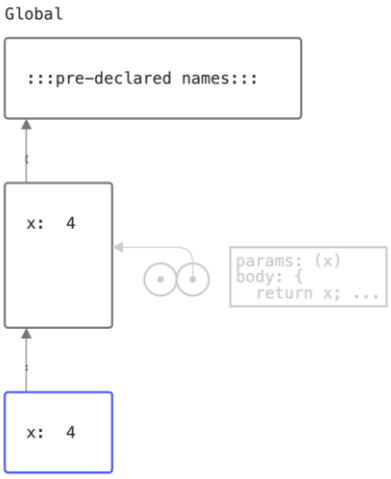
C)



D)



(22) **[3 marks] (MCQ)** Consider the following environment diagram.



Which of the following programs gives rise to this environment diagram?

(A)

```
(x => x)(x => x)(4);
```

(B)

```
let x = 4;
(x => x)(x);
```

(C)

```
function x(x) {
    return x;
}
x(4);
```

(D)

```
(x => { { let x = 4; } })(4);
```

(E)

```
let x = 4;
{
    function x(x) {
        return 4;
    }
}
```

Section G: Memoized Binary Search [9 marks]

(23) [9 marks] Consider the implementation of binary search given below. We obtain a search function for a given array `A` by applying `make_binary_search` to `A`, for example:

```
const my_search = make_binary_search(A);
```

Add lines in the provided spaces to optimize the search such that repeated applications of `my_search` to the same value `v` require only constant time, regardless of the size of `A`.

```
my_search(4711);    // 0(log n) time, where n is the size of A
my_search(4711);    // repeated call takes 0(1) time, regardless of A
```

Section H: Lazy Merge Sort [5 marks]

(24) [5 marks] (MCQ) Consider the following *stream version* of merge sort.

```
// half, rounded downwards
function middle(n) {
    return math_floor(n / 2);
}

// put the first n elements of xs into a list
function take(xs, n) {
    return (n === 0)
        ? null
        : pair(head(xs), take(tail(xs), n - 1));
}

// drop the first n elements from the list and return the rest
function drop(xs, n) {
    return (n === 0)
        ? xs
        : drop(tail(xs), n - 1);
}

function merge_streams(xs, ys) {
    if (is_null(xs)) {
        return ys;
    } else if (is_null(ys)) {
        return xs;
    } else {
        const x = head(xs);
        const y = head(ys);
        return x < y   // comparison carried out here
            ? pair(x, () => merge_streams(stream_tail(xs), ys))
            : pair(y, () => merge_streams(xs, stream_tail(ys)));
    }
}

function merge_sort_with_streams(xs) {
    if (is_null(xs)) {
        return null;
    } else if (is_null(tail(xs))) {
        return pair(head(xs), () => null);
    } else {
        const mid = middle(length(xs));
        return merge_streams(
                    merge_sort_with_streams(take(xs, mid)),
                    merge_sort_with_streams(drop(xs, mid)));
    }
}
```

Let us say we only want the first element of the sorted stream, for example:

```
const s = merge_sort_with_streams(list(8, 1, 22, …)); // 32 elements
stream_ref(s, 0); // returns the smallest element of the list
```

How many comparisons (`x < y`) are carried out by the program above, where `merge_sort_with_streams` is applied to a list with 32 numbers?

A) 5
B) 160
C) 15
D) 31
E) 63

Appendix

We assume the following pre-declared functions in Source §4 are declared as follows:

```
function map(f, xs) {
    return is_null(xs)
           ? xs
           : pair(f(head(xs)), map(f, tail(xs)));
}

function filter(pred, xs) {
    return is_null(xs)
           ? null
           : pred(head(xs))
           ? pair(head(xs), filter(pred, tail(xs)))
           : filter(pred, tail(xs));
}

function accumulate(f, initial, xs) {
    return is_null(xs)
           ? initial
           : f(head(xs), accumulate(f, initial, tail(xs)));
}

function append(xs, ys) {
    return is_null(xs)
           ? ys
           : pair(head(xs), append(tail(xs), ys));
}


function stream_map(f, s) {
    return is_null(s)
           ? null
           : pair(f(head(s)), () => stream_map(f, stream_tail(s)));
}

function stream_filter(p, s) {
    return is_null(s)
           ? null
           : p(head(s))
           ? pair(head(s), () => stream_filter(p, stream_tail(s)))
           : stream_filter(p, stream_tail(s));
}

function stream_ref(s, n) {
    return n === 0
           ? head(s)
           : stream_ref(stream_tail(s), n - 1);
}
```

```
function enum_stream(low, hi) {
    return low > hi
           ? null
           : pair(low, () => enum_stream(low + 1, hi));
}

function eval_stream(s, n) {
    return n === 0
           ? null
           : pair(head(s), eval_stream(stream_tail(s), n - 1));
}

function integers_from(n) {
    return pair(n, () => integers_from(n + 1));
}
```

——— END OF QUESTION PAPER ———